`:api-app` Firebase Crashlytics Wiring — Implementation Plan

- **Date:** 2026-06-13
- **Status:** PLAN — not implemented. Touches `google-services.json` (§6.H-sensitive) → operator approval required before any code lands.
- **Constitutional bindings:** §6.AC (Comprehensive Non-Fatal Telemetry Mandate), §6.O (Crashlytics-Resolved Issue Coverage), §6.H (Credential Security), Decoupled Reusable Architecture rule (no copy-paste between apps), §11.4.6 (no-guessing — file evidence captured; unverified items marked `UNCONFIRMED:`).
- **Investigation method:** read-only file inspection. No builds run, no secrets printed, no code edited.

---

0. Verdict (read first)

The audit's framing ("api-app has NO Crashlytics, needs operator to generate google-services.json") is PARTLY OUTDATED. Two facts change the picture:

1. **CONFIRMED — the gradle Firebase plugins are ALREADY applied to `:api-app`.** The `lava.android.application` convention plugin (`buildSrc/src/main/kotlin/AndroidApplicationConventionPlugin.kt:18-19`) applies `com.google.gms.google-services` AND `com.google.firebase.crashlytics` to *every* module using `id("lava.android.application")` — and `api-app/build.gradle.kts:21` applies exactly that. So the plugin wiring is inherited, not missing.

2. **CONFIRMED — `api-app/google-services.json` ALREADY EXISTS** (1163 bytes, dated 2026-06-02, gitignored via `.gitignore:66` `**/google-services.json`). It registers BOTH api-app application-ids:

   - `digital.vasic.lava.api` (release)
   - `digital.vasic.lava.api.dev` (debug) (Verified by grepping `package_name` keys only — NO secret values were read or printed. `mobilesdk_app_id` count = 2, matching the two app-ids.)

What is ACTUALLY missing (the real §6.AC/§6.O gap that makes runtime Crashlytics report 404 / no telemetry):

- `:api-app/build.gradle.kts` does NOT depend on the Firebase BOM / analytics / crashlytics artifacts (CONFIRMED: `grep -nE "firebase|core:common|analytics"` returns nothing).

- `ApiApplication.onCreate()` does NOT initialize Firebase (CONFIRMED: `ApiApplication.kt` only publishes `controllerHolder`/`keyStoreHolder`; no `Firebase.crashlytics`, no `FirebaseInitializer`).
- There is NO `AnalyticsTracker` binding in the api-app Hilt graph, so the embedded server's Go-side errors and the app's own catch paths have no client-side sink.

Verdict: READY TO IMPLEMENT once the operator confirms the existing `api-app/google-services.json` is the genuine console-generated file for these two app-ids (see §6 checklist item OA-1). The blocking §6.H artifact appears already present; everything else is code the agent can write. The one thing the agent cannot self-verify is whether the existing JSON is a real console export vs. a placeholder — that is an operator confirmation, not a generation task.

---

1. `:api-app` module map (evidence)

Concern	File	Notes
Gradle module decl	`settings.gradle.kts:22` `include(":api-app")`	sibling `:core:apiengine` at line 23
Build script	`api-app/build.gradle.kts`	plugins: `lava.android.application` + `lava.android.hilt`
Release app-id	`api-app/build.gradle.kts` `applicationId = "digital.vasic.lava.api"`	versionCode 8 / versionName 0.2.4, minSdk 23
Debug app-id	`applicationIdSuffix = ".dev"` → `digital.vasic.lava.api.dev`	
Manifest	`api-app/src/main/AndroidManifest.xml`	`android:name=".ApiApplication"`; foreground `ApiEngineService`; `ApiKeyProvider` (signature-permission ContentProvider)
Application class	`api-app/src/main/kotlin/lava/api/app/ApiApplication.kt`	`@HiltAndroidApp`; injects `ApiEngineController` + `ApiKeyStore`; publishes process-wide holders in `onCreate()`
Embedded server boot	`:core:apiengine` (`NativeApiEngine`) → `liblavaapi.so`	driven by `ApiEngineController` (`api-app/.../control/ApiEngineController.kt`), run inside foreground `ApiEngineService` (`.../service/ApiEngineService.kt`); APK packages the Go `.so` via `:core:apiengine`
Compose UI	`.../ui/ApiControlScreen.kt` + `ApiControlViewModel` (Orbit MVI)	

Concern	File	Notes
Challenge tests	`api-app/src/androidTest/.../challenges/Challenge0{1..5}*Test.kt`	C05 asserts embed source-hash sync

The embedded Go server's errors are already routed Go-side through `observability.RecordNonFatal` (`lava-api-go/internal/observability/nonfatal.go`) which (1) always logs structured WARNING/ERROR via OTLP, and (2) optionally posts to Firebase Crashlytics REST when `LAVA_API_FIREBASE_CRASHLYTICS_ENABLED=true`.

---

2. The `:app` Crashlytics pattern to mirror (evidence)

2a. Gradle plugins (inherited, not in `app/build.gradle.kts` directly)

`AndroidApplicationConventionPlugin.kt`:

```
apply("com.google.gms.google-services")
apply("com.google.firebase.crashlytics")
```

Plugin classpath in `buildSrc/build.gradle.kts`: `libs.firebase.crashlytics.gradlePlugin` + `libs.google.services.gradlePlugin`. Catalog (`gradle/libs.versions.toml`): `firebaseCrashlyticsGradlePlugin = "3.0.4"`, `firebaseBom = "33.15.0"`; libs `firebase-bom`, `firebase-analytics`, `firebase-crashlytics`, `firebase-perf`.

2b. Runtime deps (`app/build.gradle.kts:301-304`)

```
implementation(platform(libs.firebase.bom))
implementation(libs.firebase.analytics)
implementation(libs.firebase.crashlytics)
implementation(libs.firebase.perf)
```

2c. Application init (`app/src/main/.../LavaApplication.kt`)

- FirebaseApp is auto-initialized by `FirebaseInitProvider` (a ContentProvider the google-services plugin injects) BEFORE `Application.onCreate()` — so NO manual `initializeApp()` (manual init was a 2026-05-05 Crashlytics incident root cause).
- `LavaApplication.onCreate()` calls `FirebaseInitializer.initialize(...)` (defensively wrapped) passing nullable SDK accessors, isDebug, versionName/Code, applicationId, and a `warn` logger.
- Then `NavTeardownCrashReporter.install(analytics)` tags a known upstream crash.

2d. The telemetry surface (the §6.AC contract)

- **Interface (reusable, in `core:common`):** `lava.common.analytics.AnalyticsTracker` — `recordNonFatal(throwable, context)`, `recordWarning(message, context)`, `event/setUserId/setProperty/log`, plus `Params` (feature/module/operation/error_class/error_message/screen) + `Events`. `core/common/src/main/kotlin/lava/common/analytics/AnalyticsTracker.kt`.
- **Impl + DI + initializer (CURRENTLY app-private — `digital.vasic.lava.client.firebase`):**
  - `FirebaseAnalyticsTracker.kt` — nullable-SDK-safe; filters `CancellationException`; truncates values to 1024 chars; `recordWarning` records a synthetic `LavaNonFatalWarning`.
  - `FirebaseProvidesModule.kt` — Hilt `@InstallIn(SingletonComponent)`; `@Provides` for `FirebaseAnalytics?/FirebaseCrashlytics?/FirebasePerformance?` (all runCatching → nullable) and `analyticsTracker(...)` returning real tracker or `NoOpAnalyticsTracker`.
  - `FirebaseInitializer.kt` — `internal object`, JVM-unit-testable, per-SDK guarded.
  - `NoOpAnalyticsTracker.kt` — fallback.

---

3. Step-by-step plan to wire `:api-app`

> **Decoupled Reusable Architecture note (load-bearing design decision):** the Firebase impl/DI/initializer in `app/.../firebase/` are `internal` to `:app` and reference `digital.vasic.lava.client.BuildConfig`. Copy-pasting them into `:api-app` is a forbidden bluff vector (behaviour drifts, fixes don't propagate — the exact failure mode the rule names). **Preferred approach: extract the reusable Firebase wiring to a shared module; have BOTH `:app` and `:api-app` consume it.** A copy-paste fallback is documented but discouraged.

Option A (RECOMMENDED) — extract to a shared `:core:analytics-firebase` module

1. **Create module `core/analytics-firebase/`** applying `lava.android.library` + `lava.android.hilt`, depending on `:core:common` (for `AnalyticsTracker`) + the Firebase BOM/analytics/crashlytics deps.
2. **Move** `FirebaseAnalyticsTracker`, `FirebaseProvidesModule`, `FirebaseInitializer`, `NoOpAnalyticsTracker` into it under a neutral package (e.g. `lava.analytics.firebase`). Make `FirebaseInitializer.initialize(...)` take applicationId/versionName/versionCode as params (already does) so it is app-agnostic — no `BuildConfig` reference inside the module.
3. **`:app`** drops the 4 firebase `implementation(...)` lines + the moved files; adds `implementation(project(":core:analytics-firebase"))`. `LavaApplication` keeps calling `FirebaseInitializer.initialize(...)` with `BuildConfig.*` (now imported from the shared module).

4. **`:api-app`** adds `implementation(project(":core:analytics-firebase"))` + `implementation(project(":core:common"))`. (The Firebase BOM/deps come transitively from the shared module's `api(...)` or are re-declared as needed.)
5. `NavTeardownCrashReporter` is client-nav-specific → STAYS in `:app` (the api-app has no androidx-navigation back-stack teardown crash).

Option B (fallback, discouraged) — add deps + copy wiring into `:api-app`

- Add to `api-app/build.gradle.kts` dependencies: `implementation(platform(libs.firebase.bom))`, `firebase.analytics`, `firebase.crashlytics`, `firebase.perf`, `implementation(project(":core:common"))`.
- Recreate the 4 firebase files under `lava.api.app.firebase` referencing `lava.api.app.BuildConfig`.
- This duplicates ~250 lines and violates the spirit of the Decoupled rule; only acceptable as a time-boxed transitional step with a tracked extraction TODO.

Application init (both options) — `ApiApplication.onCreate()`

Mirror `LavaApplication` exactly:

```
@Inject lateinit var analytics: AnalyticsTracker   // from the
        Hilt graph
// ... in onCreate(), AFTER super.onCreate():
FirebaseInitializer.initialize(
    crashlytics = { runCatching {
        Firebase.crashlytics }.getOrNull() },
    analytics   = { runCatching {
        Firebase.analytics }.getOrNull() },
    performance = { runCatching {
        Firebase.performance }.getOrNull() },
    isDebug = BuildConfig.DEBUG,
    versionName = BuildConfig.VERSION_NAME,
    versionCode = BuildConfig.VERSION_CODE,
    applicationId = BuildConfig.APPLICATION_ID,
    warn = { msg, t -> Log.w(TAG, msg, t) },
)
```

Set a Crashlytics custom key distinguishing the artifact (e.g. `setCustomKey("artifact", "api-app")`) so api-app vs client crashes are separable in the shared Firebase project.

Manifest (both options)

No manifest change strictly required — `FirebaseInitProvider` is contributed by the google-services plugin's merged manifest automatically (same as `:app`). Keep `ApiApplication` as `android:name`. (`UNCONFIRMED:` whether `firebase-perf` adds a manifest provider that needs an explicit `tools:node="merge"` — `:app` builds fine without one, so api-app should too, but a build is required to prove the manifest merges cleanly; no build was run per task constraint.)

Wiring the embedded Go server's errors to client-side Crashlytics (optional, §6.AC bridge)

Two independent paths already exist; the plan only needs to pick how they reach the dashboard:

- **Go REST bridge (server-side → Firebase directly):** set `LAVA_API_FIREBASE_CRASHLYTICS_ENABLED=true` for the embedded server. The embed reads env; `ApiEngineController`/`ApiEngineService` would need to pass that env into `NativeApiEngine`. (`UNCONFIRMED:` whether the embed currently forwards arbitrary env vars to the Go runtime — needs a read of `:core:apiengine` `NativeApiEngine` env-passing surface, out of scope for this read-only pass; flag for the implementation cycle.)
- **Kotlin bridge (server-side → app `AnalyticsTracker`):** if `NativeApiEngine.status()` / the engine exposes recent errors to Kotlin, the controller can forward them via `analytics.recordNonFatal(...)` with `feature="api-embed"`. This keeps a single client-side sink and avoids shipping a Go-side Firebase API key. **RECOMMENDED over the REST bridge** because it avoids putting Firebase credentials into the Go embed (§6.H surface reduction).

---

4. §6.H / operator-action items (the blocking items)

ID	Item	Who	Status
OA-1	Confirm `api-app/google-services.json` (already present, 2026-06-02, gitignored) is the GENUINE Firebase-console export for app-ids `digital.vasic.lava.api` + `digital.vasic.lava.api.dev` — NOT a placeholder. The agent registered the package_names exist but cannot validate the API keys / mobilesdk_app_ids are real console values without reading secrets.	**Operator**	**Likely already done — needs confirmation only.**
OA-2	Confirm both api-app Firebase apps exist in the SAME Firebase project as the client apps (so api-app + client crashes share one dashboard). The `package_name` evidence shows all 4 app-ids; project-level co-location is `UNCONFIRMED:` (would require reading project_id from the JSONs — a secret-adjacent value, not read per task constraint).	**Operator**	Confirm in Firebase console.
OA-3	If the Go REST bridge path (§3) is chosen, the operator must provide the Firebase Crashlytics REST credentials/config for the embed and accept the §6.H risk of a key inside the Go `.so`. **Avoidable** by choosing the Kotlin-bridge path instead.	**Operator**	Only if REST bridge chosen.
OA-4			

ID	Item	Who	Status
	Do NOT commit either `google-services.json` (both already gitignored via `**/google-services.json`). Verify no agent step stages them.	Agent + reviewer	Enforced by `.gitignore` + pre-push.

The agent CANNOT and MUST NOT invent a `google-services.json` (§6.H). The good news: it does not need to — one already exists for the correct app-ids. The remaining operator action is **confirmation/validation**, not generation.

---

5. Anti-bluff / verification owed at implementation time (not done here)

- §6.O closure-log + validation test + Challenge for any Crashlytics issue this surfaces.
- A real-stack test that the api-app's `AnalyticsTracker` binding resolves and `recordNonFatal` reaches Crashlytics (mirror `FirebaseInitializerTest` / `FirebaseProvidesModule` coverage from `:app`).
- §6.AC: confirm every catch path in `:api-app` production code (`ApiEngineController`, `ApiEngineService`, `MdnsAdvertiser`, `ApiKeyStore`) records non-fatals — currently they have no sink.
- A build MUST be run (by the gradle-owning stream) to prove the google-services plugin accepts the existing api-app JSON and the manifest merges — this plan ran NO build per task constraint.

---

6. File evidence index (all read-only, this session)

- `settings.gradle.kts` — `:api-app` + `:core:apiengine` includes.
- `api-app/build.gradle.kts` — app-ids, plugins, deps (no firebase/ core:common).
- `api-app/src/main/AndroidManifest.xml` — `ApiApplication`, services, provider.
- `api-app/src/main/kotlin/lava/api/app/ApiApplication.kt` — no Firebase init present.
- `buildSrc/src/main/kotlin/AndroidApplicationConventionPlugin.kt` — google-services + crashlytics plugins applied to all apps.
- `buildSrc/build.gradle.kts` — firebase/google-services gradle-plugin classpath.
- `gradle/libs.versions.toml` — firebase BOM/plugin versions + libs.
- `app/build.gradle.kts:301-304` — firebase runtime deps.
- `app/src/main/.../LavaApplication.kt` — init pattern.
- `app/src/main/.../firebase/{FirebaseAnalyticsTracker,FirebaseProvidesModule,FirebaseInitializer,NoOpAnalyticsTracker}.kt` — impl/DI/initializer (app-private).
- `core/common/src/main/kotlin/lava/common/analytics/AnalyticsTracker.kt` — reusable interface.
- `lava-api-go/internal/observability/nonfatal.go` — Go-side RecordNonFatal + optional REST bridge.

- `api-app/google-services.json` + `app/google-services.json` — present, gitignored; package_names only inspected (no secret values read/printed).